

Learning Management Portal — Database Guide

Course: Database Systems | **Purpose:** Understand every database-related part of this project and prepare for the viva. Read this top-to-bottom once; you will be able to answer anything they ask.

1. The big picture — how the app talks to the database

This is a **Flask (Python) web app** backed by **PostgreSQL 16**. There is **no ORM** — we write raw SQL and run it through the `psycopg2` driver. The flow for any page is:

1. The browser hits a URL (e.g. `/courses/5`).
2. A **route** function in the `routes/` folder runs.
3. It calls `get_db_connection()` (in `db.py`) to open a PostgreSQL connection.
4. It runs SQL with `cur.execute("... %s ...", (values,))`.
5. It reads rows with `cur.fetchone()` / `cur.fetchall()`, then `conn.commit()` for writes.
6. It passes the rows to a Jinja2 HTML template and closes the connection in a `finally` block.

The one file that opens every connection — `db.py`

```
def get_db_connection():
    conn = psycopg2.connect(
        host=config.DB_HOST, port=config.DB_PORT,
        dbname=config.DB_NAME, user=config.DB_USER,
        password=config.DB_PASSWORD,
        cursor_factory=RealDictCursor,  # rows come back as Python dicts
    )
    return conn
```

Why `RealDictCursor`? It makes each row behave like a dictionary, so we write `row["course_name"]` instead of `row[1]` — readable and not order-dependent. **One connection per request**, closed in `finally`; we deliberately avoid connection pools to keep the code simple to explain.

2. The schema — 10 tables

Defined in `schema.sql`. Created in dependency order; dropped in reverse with `CASCADE` so the file re-runs cleanly.

Table	What it stores	Key relationships
users	Both students & instructors; split by role column	PK user_id
courses	One course, owned by one instructor; has a unique join_code	instructor_id → users
enrollments	Bridge: which student is in which course	→ users, → courses; UNIQUE(student, course)
assignments	Instructor's task with due date & max marks	course_id → courses
submissions	A student's uploaded file + grade + feedback	→ assignments, → users; UNIQUE(assignment, student)
quizzes	Quiz metadata (scored offline)	course_id → courses
quiz_attempts	One score per student per quiz	→ quizzes, → users
announcements	Instructor posts to a course stream	→ courses, → users
comments	A comment on EITHER a submission OR an announcement	polymorphic + CHECK constraint
grade_audit_log	Auto-written record of every grade change	filled by a trigger

Design choices they may ask about

- **One users table for both roles** — keeps every foreign key as a single user_id; the role column (with a CHECK) distinguishes them.
- **ON DELETE CASCADE everywhere** — delete a course and all its assignments, submissions, announcements vanish automatically. No orphan rows.
- **UNIQUE constraints** — email on users, join_code on courses, (student_id, course_id) on enrollments, (assignment_id, student_id) on submissions. These enforce business rules at the database level.
- **CHECK constraints** — role IN ('student', 'instructor'); grade BETWEEN 0 AND 100; the polymorphic comment rule (exactly one of submission/announcement is set).

3. The four CRUD operations — where they live

Graders will ask you to demonstrate all four. Here is a concrete example of each so you can point at code.

CREATE — making a course (routes/courses.py)

```
cur.execute(
    """INSERT INTO courses (course_name, course_description, join_code,
                           instructor_id, course_code)
       VALUES (%s, %s, %s, %s, %s) RETURNING course_id""",
    (name, description, join_code, session["user_id"], course_code),
)
new_id = cur.fetchone()["course_id"]
conn.commit()
```

`RETURNING course_id` gives us the new row's ID in the same round-trip so we can redirect to its page.

READ — listing a student's courses with live metrics

```
SELECT c.course_id, c.course_name,
       (SELECT COUNT(*) FROM assignments a WHERE a.course_id = c.course_id) AS total_assignments,
       (SELECT COUNT(*) FROM submissions s JOIN assignments a ON a.assignment_id = s.assignment_id
        WHERE a.course_id = c.course_id AND s.student_id = %s AND s.grade IS NOT NULL) AS done
FROM enrollments e JOIN courses c ON c.course_id = e.course_id
WHERE e.student_id = %s
```

The progress bar % is computed from **real data**: graded submissions ÷ total assignments. Nothing is hard-coded.

UPDATE — editing a course / grading a submission

```
UPDATE courses SET course_name = %s, course_description = %s, course_code = %s
WHERE course_id = %s
```

Grading does an `UPDATE` on `submissions.grade` — which fires the audit trigger (Section 4).

DELETE — removing a course / a comment / a student

```
DELETE FROM courses WHERE course_id = %s AND instructor_id = %s
```

Notice the extra `AND instructor_id = %s`: a user can only delete *their own* course. The same ownership pattern guards comment deletes and student removal.

4. Triggers — logic that runs automatically inside the DB

Defined in `db/triggers.sql`. A **trigger** is a function the database runs by itself when a row is inserted/updated/deleted — the app doesn't have to remember to do it.

(a) Keep `updated_at` honest — `fn_set_updated_at()`

`BEFORE UPDATE` on 5 tables, it sets `NEW.updated_at = CURRENT_TIMESTAMP`. So every edit timestamps itself automatically.

(b) Grade audit log — `fn_log_grade_changes()`

`AFTER INSERT OR UPDATE` on `submissions`. When a grade is set or changed, it writes a row into `grade_audit_log` recording the old grade, new grade, and **who** changed it. The "who" comes from a session variable the Flask code sets:

```
SET LOCAL app.current_user_id = ''; -- set by Flask in the grading transaction
-- trigger reads it back with:
current_setting('app.current_user_id', true)
```

It uses `IS DISTINCT FROM` so it does **not** log when the grade didn't actually change (and treats two `NULL`s as equal). This proves nobody silently altered a grade.

(c) Enforce enrollment on submission — `fn_enforce_enrollment_for_submission()`

`BEFORE INSERT` on `submissions`. It walks back to the assignment's course and checks the student is enrolled; if not, it `RAISE EXCEPTION` and the insert fails. This is a **database-level guard** — even a direct `psql` insert can't break the rule.

5. Views — saved queries

Defined in `db/views.sql`. A **view** is a stored SELECT you can query like a table; it avoids repeating long joins.

View	Answers the question
<code>student_grades_view</code>	"What grade did each student get on each assignment?" (joins submissions+users+assignments+courses)
<code>course_enrollment_counts</code>	"How many students are in each course?" (LEFT JOIN so empty courses show 0)
<code>instructor_dashboard</code>	"For this instructor: how many courses, students, assignments?"

6. Stored functions (PL/pgSQL)

Defined in `db/procedures.sql`. Logic kept in the database, callable from SQL.

- `fn_enroll_student(student_id, join_code)` — validates the user is a real student, the code maps to a real course, and they aren't already enrolled; then inserts the enrollment. Each failure does a clear `RAISE EXCEPTION` that Flask shows to the user.
- `fn_course_average_grade(course_id)` — returns `AVG(grade)` across graded submissions in a course (NULL if none graded yet).

7. SQL Injection — your app is SECURE

Q: What is SQL Injection?

An attack where a user types SQL fragments into an input field, and if the app builds its query by **gluing strings together**, the attacker's text becomes executable SQL. Classic example: a login form where entering `' OR '1'='1` as the password turns `WHERE password = ' OR '1'='1'` into something always true — logging in without a password. Worse payloads can read or drop tables.

Q: How do you prevent it?

Use **parameterized queries** (a.k.a. prepared statements / bound parameters). You send the SQL with `%s` placeholders and the values **separately**; the database driver treats the values strictly as data, never as SQL. Other layers: input validation, least-privilege DB user, and not

exposing raw error messages.

Q: Is THIS application vulnerable? Why / why not?

No — it is safe. Every single query in the project uses psycopg2 parameter binding:

```
cur.execute("SELECT role FROM users WHERE email = %s", (email,))
```

The %s is **not** Python string formatting — it is psycopg2's placeholder. The driver sends the query and the (email,) tuple separately, so even if someone types ' OR '1'='1 it is treated as a literal string to match, not as code. We **never** use f-strings, + concatenation, or % formatting to build SQL with user input. The stored function fn_enroll_student is also called with parameters, not concatenation.

If they push: "Show me one query and prove it's safe." → Open routes/auth.py login, point to the %s + tuple. Say: *"The value travels in a separate protocol field; PostgreSQL parses the SQL text first, then plugs the value in as data — so injected SQL is never parsed as SQL."*

8. Likely viva questions & model answers

Q1. Walk me through what happens when a student submits an assignment.

The route receives the uploaded file, saves it under `uploads/submissions/`, then runs an `INSERT INTO submissions` (or `UPDATE` if re-submitting — enforced by the `UNIQUE(assignment_id, student_id)` constraint). The `BEFORE INSERT` trigger checks the student is enrolled; if not, the insert is rejected. We then commit.

Q2. How do you store passwords?

We never store plaintext. We hash with **bcrypt** (a slow, salted hashing algorithm) and store only the hash in `users.password_hash`. At login we use bcrypt's check function to compare. bcrypt is deliberately slow to resist brute-force, and salts defeat rainbow tables.

Q3. Why use foreign keys with `ON DELETE CASCADE`?

Foreign keys enforce referential integrity — you can't have a submission pointing to a non-existent assignment. `CASCADE` means deleting a parent (e.g. a course) auto-deletes its children (assignments, enrollments, announcements), so we never leave orphan rows or have to delete them manually in code.

Q4. Difference between a view, a trigger, and a stored function here?

A **view** is a saved `SELECT` (read-only convenience). A **function** is callable logic that returns a value (e.g. enrol a student, compute an average). A **trigger** is a function the DB runs *automatically* on insert/update/delete (e.g. audit a grade change). We use all three.

Q5. How do you guarantee a student can't enrol twice / submit twice?

Database `UNIQUE` constraints: `UNIQUE(student_id, course_id)` on enrollments and `UNIQUE(assignment_id, student_id)` on submissions. The DB rejects the duplicate even if the app logic had a bug.

Q6. What is a transaction and where do you use one?

A transaction groups statements so they all succeed or all roll back. We `conn.commit()` on success and `conn.rollback()` in the `except` block. Grading uses one transaction that sets `app.current_user_id` and updates the grade together, so the audit trigger records the correct user.

Q7. Why no ORM?

For a DB-systems course we wanted to write and own the SQL — joins, constraints, triggers, views — rather than hide them behind an ORM. It makes the database concepts visible and gradeable.

Q8. What does RealDictCursor do?

It returns each row as a dictionary keyed by column name, so we access `row["grade"]` instead of a positional index. Cleaner and safe against column-order changes.

30-second summary to memorise: "It's a Flask + PostgreSQL app, raw SQL via psycopg2, 10 tables with FK CASCADE and UNIQUE/CHECK constraints. We use views for reports, PL/pgSQL functions for enrolment and averages, and triggers for grade auditing, timestamps, and enrolment enforcement. All queries are parameterized so we're safe from SQL injection. Passwords are bcrypt-hashed."